Problem1: The Need For Speed: Vectorization and Numpy

Performing mathematical operations on vectors and matrices is ubiquitous in most machine learning algorithms. Whether it's a simple similarity measure that works by calculating the dot product between two vectors, or deep neural networks, they all involve repeated matrix operations. This makes it imperative that our underlying code design performs matrix operations efficiently.

1.1 The Perils of Python

While Python is widely the language of choice for machine learning researchers across the globe (thanks to the speed of development and code readability it offers and the support it enjoys from the open-source community), Python as a high-level language on average is much slower than a lower level language like C++. To combat this, libraries like numpy and scipy implement most of the back-end operations they perform in C / C++, while providing wrappers in Python to be able to call underlying C code seamlessly from a Python script.

1.2 Speed Comparison: Numpy and Python

We highly recommend you to use numpy extensively in this course, it will be difficult to pass the programming portion of Homework 4 without writing most of your matrix operations in numpy. In this section, we'll see why.

Consider you have two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$. To see how similar they are, as measured by the cosine angle between them, you want to compute their dot product. This translates to the following operation:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \ldots + a_n b_n$$

When translated to code, notice how the dot product in NumPy is a whopping 100x faster than the native Python!

In [2]:
```
from timeit import timeit
import numpy as np
import array

VECTOR_SIZE = int(1e8)

# NumPy arrays
a = np.random.rand(VECTOR_SIZE)
b = np.random.rand(VECTOR_SIZE)

# Python arrays
aArr = array.array('d', a)
bArr = array.array('d', b)
```

```
def test_py_arr():
    sum = 0
    for i in range(VECTOR_SIZE):
        # TODO: apply dot product by using aArr and bArr
        sum += aArr[i] * bArr[i]


    return sum

def test_np():
    # TODO: apply dot product by using a, b. Using numpy to vectorize to
    return np.dot(a, b)


# faster than multiprocessing , python lists , or numpy arrays with pytho
# faster than using a range and indexing

def time_dot_product (f):
    return timeit(f, setup=f, number =5) / 5
```

Test to time consumption! 😊

In [3]:
```
print (f" NumPy = {time_dot_product (test_np) :.2f}s") # about 0.05 s
print (f" Python on an array = {time_dot_product(test_py_arr) :.2f}s") #
```

```
NumPy = 0.05s
Python on an array = 12.62s
```

Problem2: k-Nearest Neighbor (kNN) exercise

Complete and hand in this completed worksheet (including its outputs and any supporting code outside of the worksheet) with your assignment submission.

The kNN classifier consists of two stages:

- During training, the classifier takes the training data and simply remembers it
- During testing, kNN classifies every test image by comparing to all training images and transfering the labels of the k most similar training examples
- The value of k is cross-validated

In this exercise you will implement these steps and understand the basic Image Classification pipeline, cross-validation, and gain proficiency in writing efficient, vectorized code.

In [4]:
```
# Run some setup code for this notebook.

import random
import numpy as np
from data_utils import load_CIFAR10
import matplotlib.pyplot as plt

# This is a bit of magic to make matplotlib figures appear inline in the
```

```
# rather than in a new window.
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0) # set default size of plots
plt.rcParams['image.interpolation'] = 'nearest'
plt.rcParams['image.cmap'] = 'gray'

# Some more magic so that the notebook will reload external python module
# see http://stackoverflow.com/questions/1907993/autoreload-of-modules-in
%load_ext autoreload
%autoreload 2
```

In [5]:
```
# Load the raw CIFAR-10 data.
cifar10_dir = 'datasets/cifar-10-batches-py'

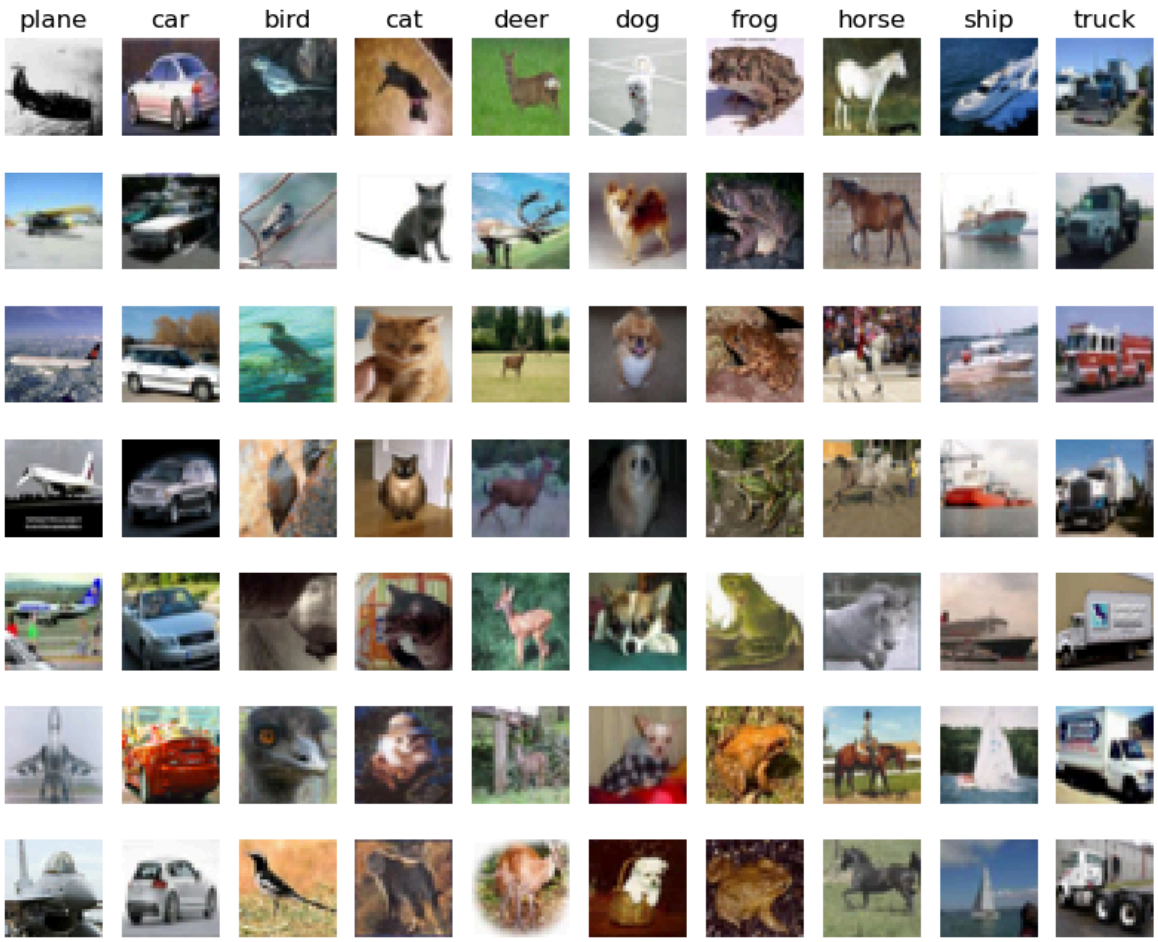
# Cleaning up variables to prevent loading data multiple times (which may
try:
   del X_train, y_train
   del X_test, y_test
   print('Clear previously loaded data.')
except:
   pass

X_train, y_train, X_test, y_test = load_CIFAR10(cifar10_dir)

# As a sanity check, we print out the size of the training and test data.
print('Training data shape: ', X_train.shape)
print('Training labels shape: ', y_train.shape)
print('Test data shape: ', X_test.shape)
print('Test labels shape: ', y_test.shape)
```

```
Training data shape:  (50000, 32, 32, 3)
Training labels shape:  (50000,)
Test data shape:  (10000, 32, 32, 3)
Test labels shape:  (10000,)
```

In [6]:
```
# Visualize some examples from the dataset.
# We show a few examples of training images from each class.
classes = ['plane', 'car', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse',
num_classes = len(classes)
samples_per_class = 7
for y, cls in enumerate(classes):
    # print(y, cls)
    idxs = np.flatnonzero(y_train == y)
    idxs = np.random.choice(idxs, samples_per_class, replace=False)
    for i, idx in enumerate(idxs):
        plt_idx = i * num_classes + y + 1
        plt.subplot(samples_per_class, num_classes, plt_idx)
        plt.imshow(X_train[idx].astype('uint8'))
        plt.axis('off')
        if i == 0:
            plt.title(cls)
plt.show()
```



In [7]:
```
# Subsample the data for more efficient code execution in this exercise
num_training = 5000
mask = list(range(num_training))
X_train = X_train[mask]
y_train = y_train[mask]

num_test = 500
mask = list(range(num_test))
X_test = X_test[mask]
y_test = y_test[mask]

print('X_train origin shape : ', X_train.shape)
# Reshape the image data into rows
X_train = np.reshape(X_train, (X_train.shape[0], -1))
X_test = np.reshape(X_test, (X_test.shape[0], -1))
print(X_train.shape, X_test.shape)
```

```
X_train origin shape :  (5000, 32, 32, 3)
(5000, 3072) (500, 3072)
```

In [8]:
```
from k_nearest_neighbor import KNearestNeighbor

# Create a kNN classifier instance.
# Remember that training a kNN classifier is a noop:
# the Classifier simply remembers the data and does no further processing
classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
```

We would now like to classify the test data with the kNN classifier. Recall that we can break down this process into two steps:

1. First we must compute the distances between all test examples and all train examples.
2. Given these distances, for each test example we find the k nearest examples and have them vote for the label

Lets begin with computing the distance matrix between all training and test examples. For example, if there are **Ntr** training examples and **Nte** test examples, this stage should result in a **Nte x Ntr** matrix where each element (i,j) is the distance between the i-th test and j-th train example.

Note: For the three distance computations that we require you to implement in this notebook, you may not use the np.linalg.norm() function that numpy provides. Instead, you should just think about the defination of 'norm' and implement it by yourself.

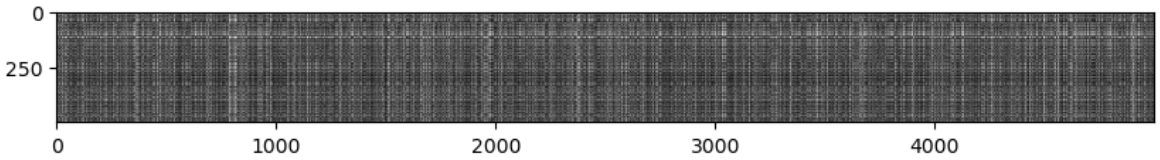
First, open `k_nearest_neighbor.py` and implement the function `compute_distances_two_loops` that uses a (very inefficient) double loop over all pairs of (test, train) examples and computes the distance matrix one element at a time.

In [9]:
```
# Open hw2_coding/k_nearest_neighbor.py and implement
# compute_distances_two_loops.

# Test your implementation:
dists = classifier.compute_distances_two_loops(X_test)
print(dists.shape)
```

```
(500, 5000)
```

In [10]:
```
# We can visualize the distance matrix: each row is a single test example
# its distances to training examples
plt.imshow(dists, interpolation='none')
plt.show()
```



Inline Question 1

Notice the structured patterns in the distance matrix, where some rows or columns are visibly brighter. (**Note that with the default color scheme black indicates low distances while white indicates high distances.**)

- What in the data is the cause behind the distinctly bright rows?
- What causes the columns?

Your Answer :

- The distinctly bright rows are caused by the test images that are very different from all the training images. This means that the test image is very different from

all the training images, and hence the distance between the test image and all the training images is very high. This is why the row is bright.

- The distinctly bright columns are caused by the training images that are very different from all the test images. This means that the training image is very different from all the test images, and hence the distance between the training image and all the test images is very high. This is why the column is bright.

Now implement the function `predict_labels` in `k_nearest_neighbor.py`, and run the code below:

In [11]:
```
# We use k = 1 (which is Nearest Neighbor).
y_test_pred = classifier.predict_labels(dists, k=1)

# Compute and print the fraction of correctly predicted examples
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, acc
```

```
Got 137 / 500 correct => accuracy: 0.274000
```

You should expect to see approximately `27%` accuracy. Now lets try out a larger `k`, say `k = 5`:

In [12]:
```
y_test_pred = classifier.predict_labels(dists, k=5)
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, acc
```

```
Got 139 / 500 correct => accuracy: 0.278000
```

You should expect to see a slightly better performance than with `k = 1`.

In the following parts of this section, please open `k_nearest_neighbor.py`, then implement the functions `compute_distances_one_loop`, and `compute_distances_no_loops`.

In [13]:
```
# Now lets speed up distance matrix computation by using partial vectoriz
# with one loop. Implement the function compute_distances_one_loop and ru
# code below:
dists_one = classifier.compute_distances_one_loop(X_test)

# To ensure that our vectorized implementation is correct, we make sure t
# agrees with the naive implementation. There are many ways to decide whe
# two matrices are similar; one of the simplest is the Frobenius norm. In
# you haven't seen it before, the Frobenius norm of two matrices is the s
# root of the squared sum of differences of all elements; in other words,
# the matrices into vectors and compute the Euclidean distance between th
difference = np.linalg.norm(dists - dists_one, ord='fro')
print('One loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
One loop difference was: 0.000000
Good! The distance matrices are the same
```

In [14]:
```
# Now implement the fully vectorized version inside compute_distances_no_
# and run the code
dists_two = classifier.compute_distances_no_loops(X_test)

# check that the distance matrix agrees with the one we computed before:
difference = np.linalg.norm(dists - dists_two, ord='fro')
print('No loop difference was: %f' % (difference, ))
if difference < 0.001:
    print('Good! The distance matrices are the same')
else:
    print('Uh-oh! The distance matrices are different')
```

```
No loop difference was: 0.000000
Good! The distance matrices are the same
```

In [15]:
```
# Let's compare how fast the implementations are
def time_function(f, *args):
    """
    Call a function f with args and return the time (in seconds) that it
    """
    import time
    tic = time.time()
    f(*args)
    toc = time.time()
    return toc - tic

two_loop_time = time_function(classifier.compute_distances_two_loops, X_t
print('Two loop version took %f seconds' % two_loop_time)

one_loop_time = time_function(classifier.compute_distances_one_loop, X_te
print('One loop version took %f seconds' % one_loop_time)

no_loop_time = time_function(classifier.compute_distances_no_loops, X_tes
print('No loop version took %f seconds' % no_loop_time)

# You should see significantly faster performance with the fully vectoriz

# NOTE: depending on what machine you're using,
# you might not see a speedup when you go from two loops to one loop,
# and might even see a slow-down.
```

```
Two loop version took 42.443606 seconds
One loop version took 64.738191 seconds
No loop version took 0.230732 seconds
```

Cross-validation

We have implemented the k-Nearest Neighbor classifier but we set the value k = 5 arbitrarily. We will now determine the best value of this hyperparameter with cross-validation.

In [18]:
```
num_folds = 5
k_choices = [1, 3, 5, 8, 10, 12, 15, 20, 50, 100] # Stored all possible c

X_train_folds = []
y_train_folds = []
################################################################################
# TODO:
```

```
# Split up the training data into folds. After splitting, X_train_folds a
# y_train_folds should each be lists of length num_folds, where
# y_train_folds[i] is the label vector for the points in X_train_folds[i]
# Hint: Look up the numpy array_split function.
################################################################################
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

X_train_folds = np.array_split(X_train, num_folds)
y_train_folds = np.array_split(y_train, num_folds)

# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

# A dictionary holding the accuracies for different values of k that we f
# when running cross-validation. After running cross-validation,
# k_to_accuracies[k] should be a list of length num_folds giving the diff
# accuracy values that we found when using that value of k.
k_to_accuracies = {}

################################################################################
# TODO:
# Perform k-fold cross validation to find the best value of k. For each
# possible value of k, run the k-nearest-neighbor algorithm num_folds tim
# where in each case you use all but one of the folds as training data an
# last fold as a validation set. Store the accuracies for all fold and al
# values of k in the k_to_accuracies dictionary.
################################################################################
# *****START OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

for k in k_choices:
    k_to_accuracies[k] = []
    for i in range(num_folds):
        X_train_fold = np.concatenate([X_train_folds[j] for j in range(nu
        y_train_fold = np.concatenate([y_train_folds[j] for j in range(nu
        X_val_fold = X_train_folds[i]
        y_val_fold = y_train_folds[i]

        classifier.train(X_train_fold, y_train_fold)
        dists = classifier.compute_distances_no_loops(X_val_fold)
        y_val_pred = classifier.predict_labels(dists, k=k)

        num_correct = np.sum(y_val_pred == y_val_fold)
        accuracy = float(num_correct) / len(y_val_fold)
        k_to_accuracies[k].append(accuracy)

# *****END OF YOUR CODE (DO NOT DELETE/MODIFY THIS LINE)*****

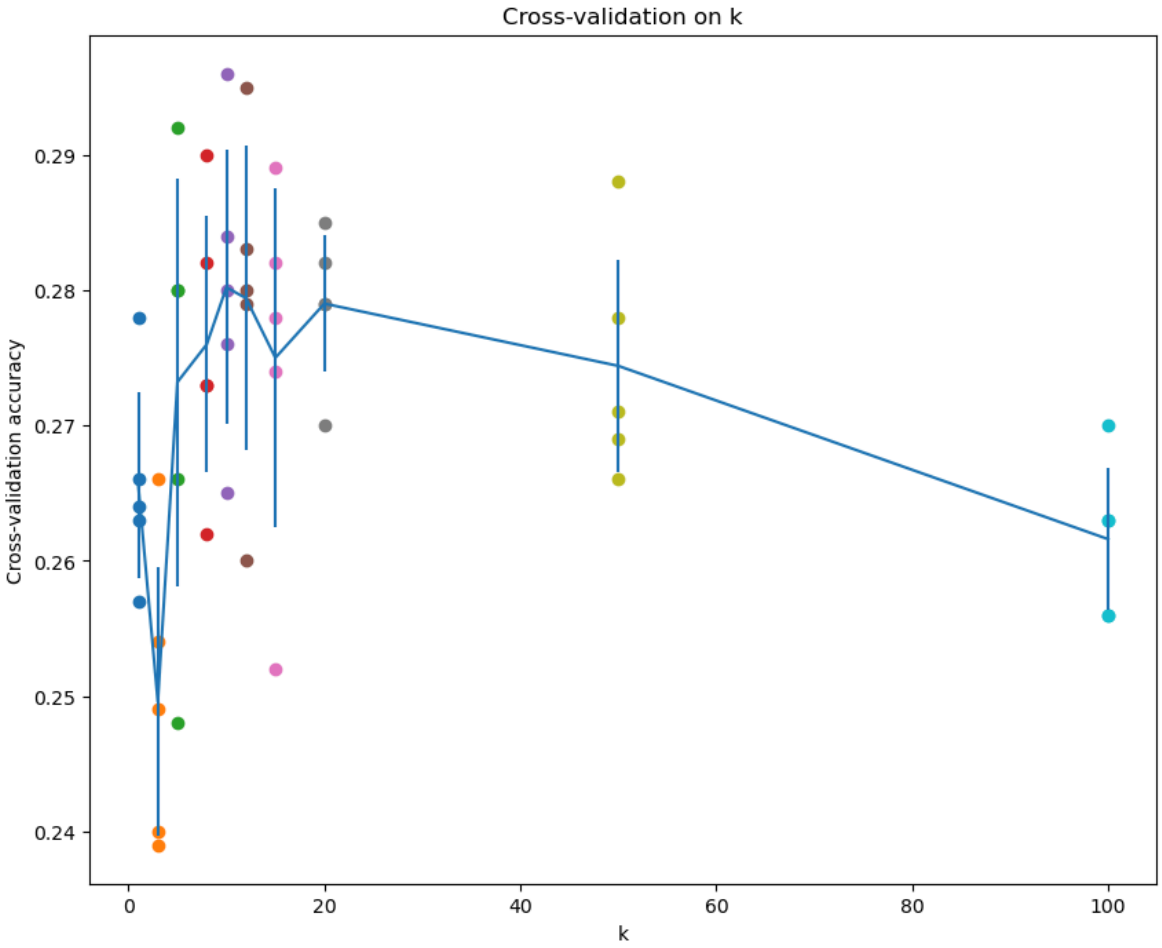
# Print out the computed accuracies
for k in sorted(k_to_accuracies):
    for accuracy in k_to_accuracies[k]:
        print('k = %d, accuracy = %f' % (k, accuracy))
```

```
k = 1, accuracy = 0.263000
k = 1, accuracy = 0.257000
k = 1, accuracy = 0.264000
k = 1, accuracy = 0.278000
k = 1, accuracy = 0.266000
k = 3, accuracy = 0.239000
k = 3, accuracy = 0.249000
k = 3, accuracy = 0.240000
k = 3, accuracy = 0.266000
k = 3, accuracy = 0.254000
k = 5, accuracy = 0.248000
k = 5, accuracy = 0.266000
k = 5, accuracy = 0.280000
k = 5, accuracy = 0.292000
k = 5, accuracy = 0.280000
k = 8, accuracy = 0.262000
k = 8, accuracy = 0.282000
k = 8, accuracy = 0.273000
k = 8, accuracy = 0.290000
k = 8, accuracy = 0.273000
k = 10, accuracy = 0.265000
k = 10, accuracy = 0.296000
k = 10, accuracy = 0.276000
k = 10, accuracy = 0.284000
k = 10, accuracy = 0.280000
k = 12, accuracy = 0.260000
k = 12, accuracy = 0.295000
k = 12, accuracy = 0.279000
k = 12, accuracy = 0.283000
k = 12, accuracy = 0.280000
k = 15, accuracy = 0.252000
k = 15, accuracy = 0.289000
k = 15, accuracy = 0.278000
k = 15, accuracy = 0.282000
k = 15, accuracy = 0.274000
k = 20, accuracy = 0.270000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.279000
k = 20, accuracy = 0.282000
k = 20, accuracy = 0.285000
k = 50, accuracy = 0.271000
k = 50, accuracy = 0.288000
k = 50, accuracy = 0.278000
k = 50, accuracy = 0.269000
k = 50, accuracy = 0.266000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.270000
k = 100, accuracy = 0.263000
k = 100, accuracy = 0.256000
k = 100, accuracy = 0.263000
```

In [19]:
```
# plot the raw observations
for k in k_choices:
    accuracies = k_to_accuracies[k]
    plt.scatter([k] * len(accuracies), accuracies)

# plot the trend line with error bars that correspond to standard deviati
accuracies_mean = np.array([np.mean(v) for k,v in sorted(k_to_accuracies.
accuracies_std = np.array([np.std(v) for k,v in sorted(k_to_accuracies.it
plt.errorbar(k_choices, accuracies_mean, yerr=accuracies_std)
```

```
plt.title('Cross-validation on k')
plt.xlabel('k')
plt.ylabel('Cross-validation accuracy')
plt.show()
```



In [20]:
```
# Based on the cross-validation results above, choose the best value for
# retrain the classifier using all the training data, and test it on the
# data. You should be able to get above 28% accuracy on the test data.

best_k = 10
# from the graph above, we could see that when k = 10, it has the best re

classifier = KNearestNeighbor()
classifier.train(X_train, y_train)
y_test_pred = classifier.predict(X_test, k=best_k)

# Compute and display the accuracy
num_correct = np.sum(y_test_pred == y_test)
accuracy = float(num_correct) / num_test
print('Got %d / %d correct => accuracy: %f' % (num_correct, num_test, acc
```

```
Got 141 / 500 correct => accuracy: 0.282000
```

Inline Question 2

Which of the following statements about k-Nearest Neighbor (k-NN) are true in a classification setting, and for all k? Select all that apply.

1. The decision boundary of the k-NN classifier is linear.
2. The training error of a 1-NN will always be lower than or equal to that of 5-NN.

3. The test error of a 1-NN will always be lower than that of a 5-NN.
4. The time needed to classify a test example with the k-NN classifier grows with the size of the training set.
5. None of the above.

Your Answer :

1. False
2. True
3. False
4. True
5. False

Your Explanation :

1. The decision boundary of the k-NN classifier is determined by the distance between the test example and the training examples, and is not a linear boundary.
2. The training error of a 1-NN will always be lower than or equal to that of 5-NN. This is because the 1-NN classifier will always have a training error of 0, as it will always classify the training data correctly. The 5-NN classifier will have a training error that is greater than or equal to the 1-NN classifier, as it will classify the training data correctly, but it may not classify it as accurately as the 1-NN classifier.
3. The test error of a 1-NN will not always be lower than that of a 5-NN. This is because the 1-NN classifier will overfit the training data, and may not generalize well to the test data. The 5-NN classifier will generalize better to the test data, and may have a lower test error than the 1-NN classifier.
4. The time needed to classify a test example with the k-NN classifier grows with the size of the training set. This is because the k-NN classifier needs to calculate the distance between the test example and all the training examples, and as the training set grows, the number of distances that need to be calculated grows as well. This will increase the time needed to classify a test example with the k-NN classifier.
5. As shown above, statements 2 and 4 are true.